Advanced Stock Market Prediction Deep Learning Reproducibility

Group 1

Christian Adcock | Daniel Guthrie | Donovan Manogue | Ethan Stanks

The stock market is a game many people wish they can play. Some make it big, lose it all, or play it safe. What if we can use historical ticker data to forecast future closing prices using deep learning. The paper, *Advanced Stock Market Prediction Using Long Short-Term Memory Networks: A Comprehensive Deep Learning Framework*, does just that. The paper uses an LSTM model to predict closing stock prices for four large U.S. companies: Apple, Amazon, Google, and Microsoft. Not only does their model use historical ticker data, but also sentiment scores based on news and social media text. For this report, we will focus on rebuilding the data setup, visual graphs, sequence creation, model training, and model evaluation scores for each of the four tickers. All steps are documented to show where reproducing went right and wrong along the way using the paper and its code appendix.

The methodology approach we are taking is following the code appendix the original authors outlined in the paper. The paper itself did not include a github repository, version requirements, raw data files, or randomness seeds. The code appendix includes python libraries such as yfinance (ticker data source), numpy, pandas, sklearn, tensorflow keras, and vaderSentiment (news data processing). The code appendix also includes the model pipeline the paper used. Our model architecture mirrored the design of the paper, utilizing stacked LSTM layers, dropout regularization, and a dense output layer. Based on the test data used in the paper, spanning from February to April 2025, our team decided on using Python version 3.12.9. This python version was the latest release during February, 2025. As for randomness, we decided on locking it to seed 42. This is a very common seed used throughout the field so the odds are in our favor, unless the authors went against the data science norm of defining a seed. Now the tricky part is finding out the python library version. They not only have to be compatible with our Python version, but also need to work alongside each other. The sweet spots we found was:

- numpy==1.26.4
- pandas==2.2.2
- scipy==1.13.1
- scikit-learn==1.5.1
- matplotlib==3.8.4
- seaborn==0.13.2
- yfinance==1.2.0
- vaderSentiment==3.3.2
- tensorflow==2.16.2
- ipykernel==6.29.5

With our environment locked down we were able to start working towards reproducing the paper's data. The data used comes from two sources. Yahoo finance for historical stock ticker data. Bloomberg and Reuters for financial news articles and headlines. Data processing itself for stock ticker is pretty simple. The code appendix used the yfinance Python library. This library acts as an API with easy to use function calls to query historical data from Yahoo Finance.

Simply supplying the ticker, start date, and end date from the range we want to look at will supply us with all the ticker data needed to reproduce the findings. Using the library made the data ready to use with no missing data or extra cleaning needed. The closing price from this data served as the target variable, and a 60-day sliding window was applied to create sequences for the LSTM model, consistent with the framework described in the original research. As for the news data, that is a different story. There is exactly 267 days of news data we'd have to collect. The paper doesn't say if they collected these headlines as the top news article for that particular day or the top article regarding that specific ticker. The methodology we tried were different news APIs. We will go into the challenges faced on that later on.

Now that our environment reproduced, we each tried to replicate the paper's findings on each of the four tickers used in their training, testing, and evaluation. The original performance metrics the paper found across each of the technology stocks are:

Stock	MAE	MSE	RMSE	MAPE (%)
Apple	6.12	58.03	7.62	2.72
Google	5.89	52.14	7.22	2.65
Microsoft	6.45	60.27	7.76	2.91
Amazon	6.78	65.43	8.09	3.05

With these results in mind, we can see how close we were to reproducing their evaluation, following the code appendix. We also set a small goal to try and recreate some of the many graphs the paper includes.

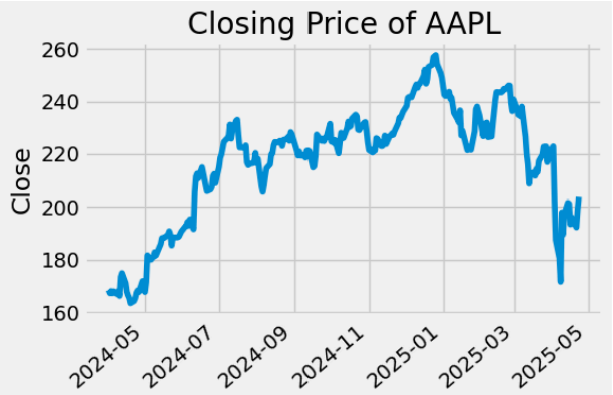
<u>Apple's Ticker Reproducibility:</u>

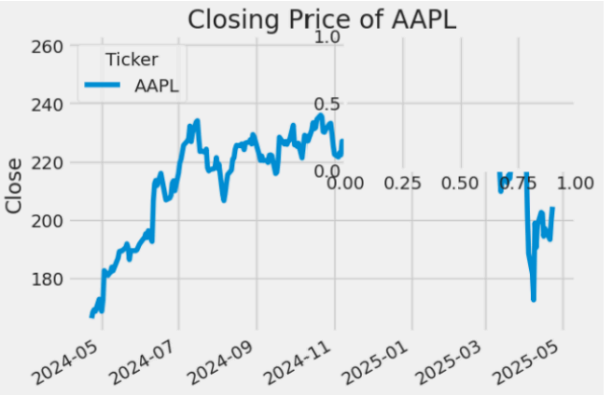
For Apple (AAPL) stock, the reproduction attempt followed the methodology outlined in the original paper, utilizing historical stock price data obtained through the yfinance library and implementing the LSTM-based architecture described in the code appendix. The data was collected for the time period from April 2024 to April 2025, and the closing price was used as the target variable. A 60-day sliding window was applied to generate sequences, and the model was trained using an 80-20 train-test split, consistent with the original study. Due to the lack of accessible historical news data, a placeholder sentiment approach was used, resulting in neutral sentiment values across the dataset.

When comparing results to the original paper, the reproduced Apple model did not match the reported performance metrics. The original paper reported the following values for Apple: MAE = 6.12, MSE = 58.03, RMSE = 7.62, and MAPE = 2.72% . In contrast, the reproduced model produced higher error values across all evaluation metrics. Specifically, the reproduced Apple model resulted in MAE = 12.9504, MSE = 284.2601, RMSE = 16.8600, and MAPE = 6.3495%.

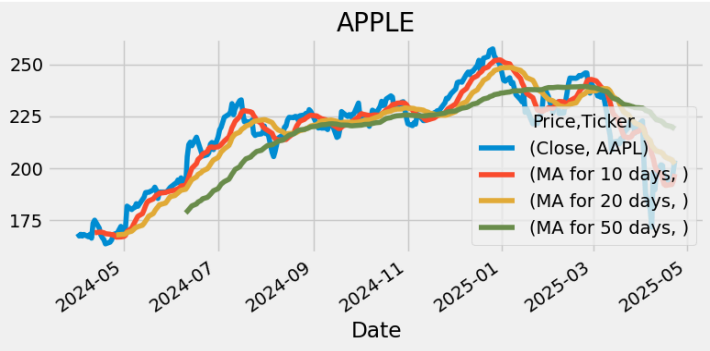
MAE	MSE	RMSE	MAPE (%)
12.9504	284.2601	16.8600	6.3495%

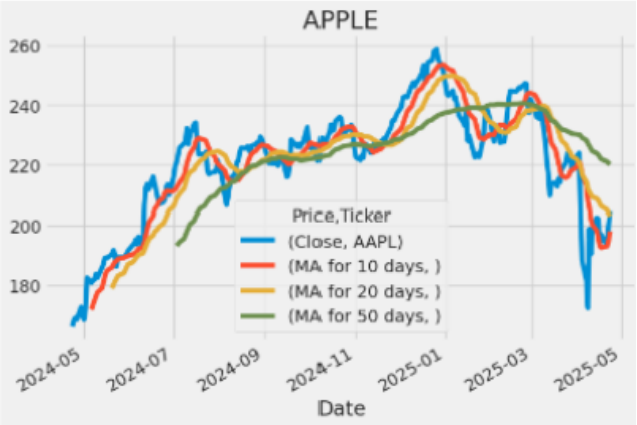
The differences between the reproduced and original results are substantial. The MAE increased by approximately 6.83, the MSE increased by 226.23, the RMSE increased by 9.24, and the MAPE increased by 3.63 percentage points.





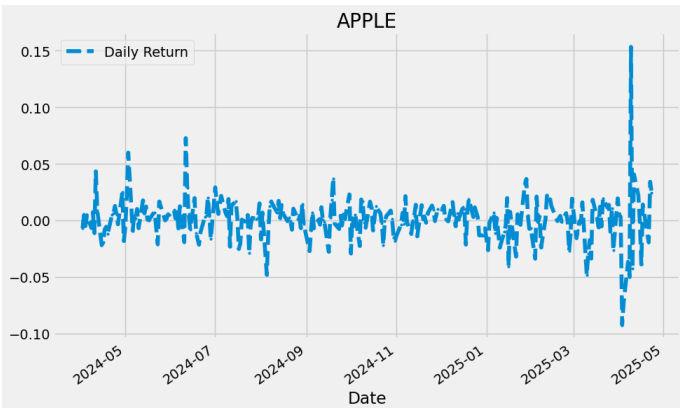
On the left is a recreation of the paper's Apple closing price visualization (Figure 2 in the original paper). The recreated graph closely matches the overall trend shown in the paper, with a steady increase through mid-to-late 2024, followed by a peak in early 2025 and a decline afterward. The shape, volatility, and general movement of the stock align well with the original figure, indicating that the data collection and preprocessing steps were successfully reproduced.

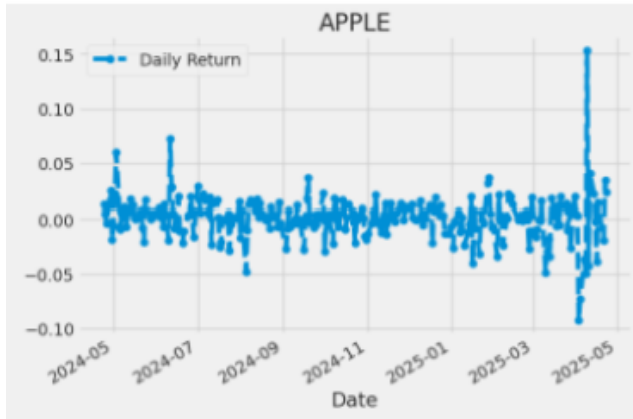




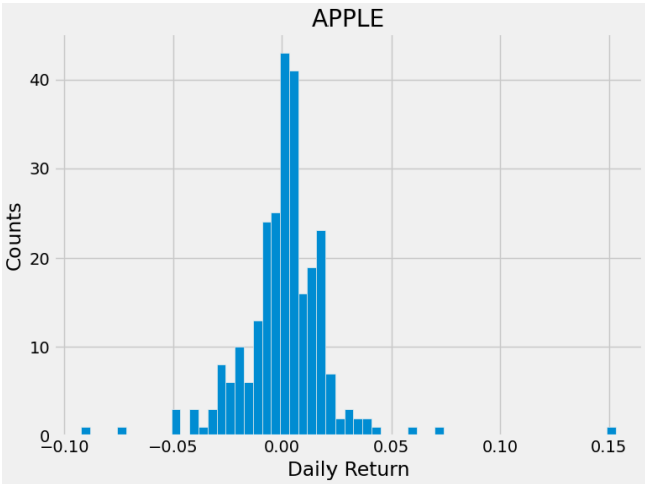
On the left is a recreation of the paper's moving average visualization (Figure 6), which includes the 10-day, 20-day, and 50-day moving averages. The recreated graph closely mirrors the

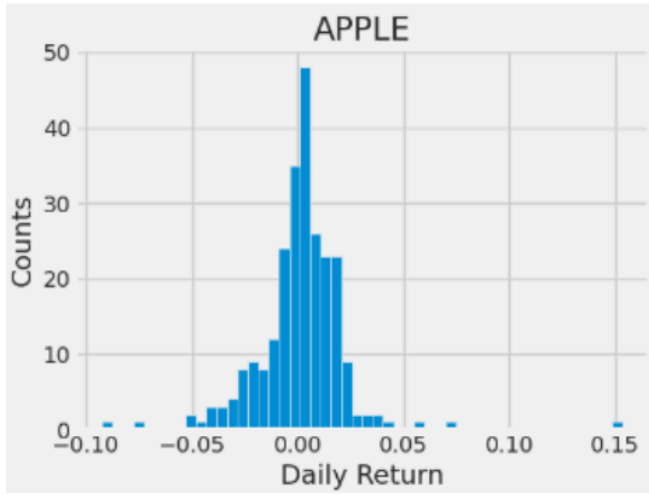
structure of the original, with the shorter moving averages reacting more quickly to price fluctuations and the 50-day moving average providing a smoother long-term trend. The alignment of these curves with the closing price confirms that the feature engineering process was implemented correctly.





On the left is a recreation of the paper's Apple daily return plot (Figure 12). The recreated graph shows daily returns centered around zero with occasional spikes, reflecting periods of higher volatility. This matches the behavior shown in the original paper, where return series are used to analyze stock volatility and temporal fluctuations.





Lastly, on the left is a recreation of the paper's histogram of Apple daily returns (Figure 13). The recreated histogram is approximately normally distributed and centered around zero, with a few outliers on both ends. This closely matches the distribution shown in the original paper, although the exact binning may differ slightly due to the paper not specifying the number of bins used.

<u>Microsoft's Ticker Reproducibility:</u>

For Microsoft (MSFT) stock, the reproduction attempt followed the methodology outlined in the original paper, combining historical stock price data with sentiment analysis to improve prediction accuracy. In this reproduction, sentiment data was obtained using the Alpha Vantage NEWS_SENTIMENT API. Sentiment scores were aggregated daily and aligned with stock price dates to form a two-feature input consisting of both price and sentiment. The API successfully returned sentiment-scored articles; however, the dataset included not only Microsoft-specific news but also indirectly related articles, such as ETF holdings, sector-wide updates, and broader technology news. This introduced noise into the sentiment feature, as not all articles were directly connected to Microsoft's performance. When comparing results, the reproduced model for Microsoft did not perfectly match the performance reported in the paper. The MSFT ticker results showed differences in Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and MAPE.

MAE	MSE	RMSE	MAPE (%)
9.74	153.18	12.38	2.50%

Although the general trend of the predictions aligned with actual stock prices, the numerical metrics did not match exactly. This discrepancy may be attributed to several factors, including differences in sentiment data quality, the limited number of sentiment observations, potential variations in hyperparameters that were not fully detailed in the paper, and inherent randomness during model training. Despite these differences, the reproduction was partially successful, as the overall modeling pipeline and predictive behavior were consistent with the original study.

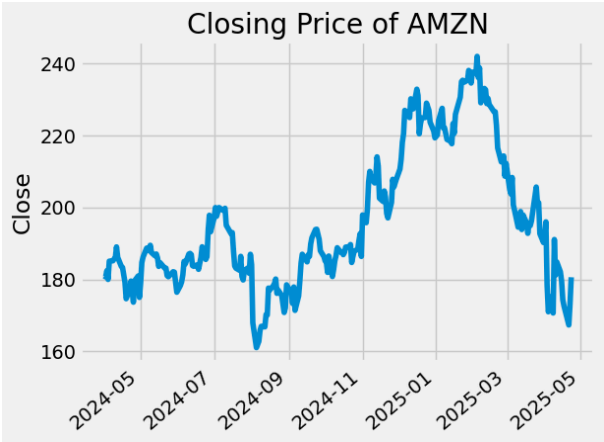
<u>Amazon's Ticker Reproducibility:</u>

For Amazon (AMZN) stock, the reproduction attempt, like the others, followed the modeling found in the paper's code appendix. Using yfinance, data was loaded for AMNZ from April 1, 2024 to April 24, 2025. Due to not having the original news data, the AMNZ model uses "Sample news article text", the example data outlined in the code appendix, for each day's news sentiment. This resulted in a sentiment score of 0.0 for every single day in the data. Researchers should have taken a 'screenshot' of their news data and included it with their paper for future researchers to evaluate their exact findings. Because of this, the AMZN model compared horribly to the original results. Using the same pipeline structure; SentimentIntensityAnalyzer > MinMaxScaler > 60 Day Sequence > 80-20 train test ratio > LSTM(64) + Dropout(0.2) + LSTM(32) + Dense(1) with a loss of mse, the model scored:

MAE	MSE	RMSE	MAPE (%)
0.0934	0.0122	0.1103	38.1549%

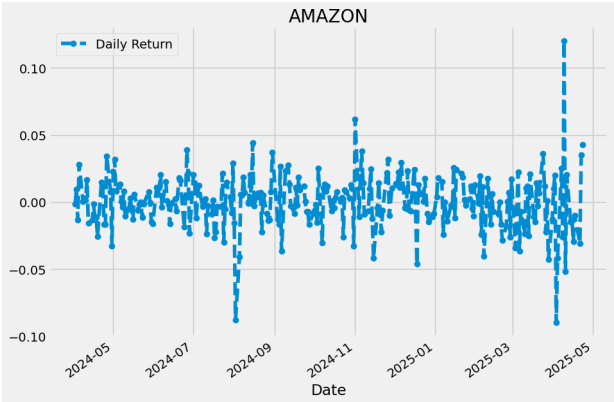
To compare, the MAE difference is -6.6865, MSE difference is -65.4178, RMSE is -7.9797, and MAPE difference is 35.1048%. Not at all close to the original findings. That is because their model is backed by the news sentiment data. Without it, no research will be able to get

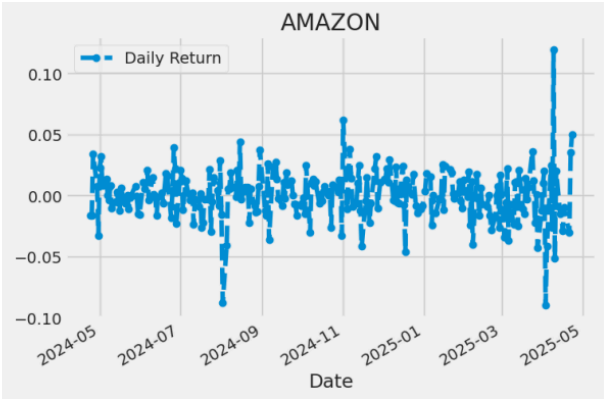
anywhere close to their original results. However, we were able to nearly make identical visualizations used in the paper.



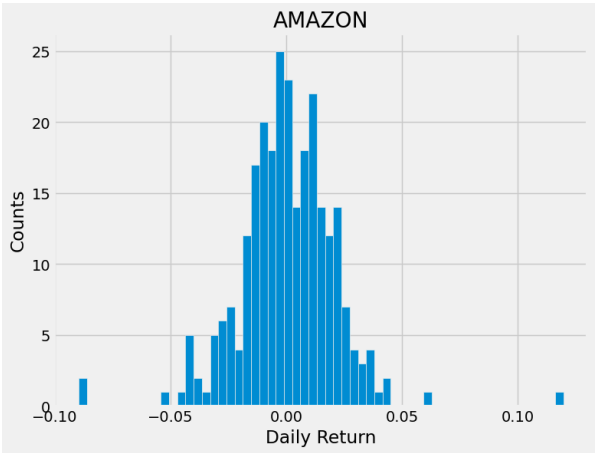
On the left, is a recreation of the paper's AMZN closing price visualization. On the right, is the original. The code appendix shows a date range for the ticker data from April 1, 2024 to April 1, 2025. When recreating this graph, I noticed we were missing data. Playing around I found that April 24, 2025 was the closest match to their end date. As of writing this, I notice that their data must start around April 24, 2025 as well, as the image on the left has data not expressed in the original. During the AMZN reproduction, we did not catch that.

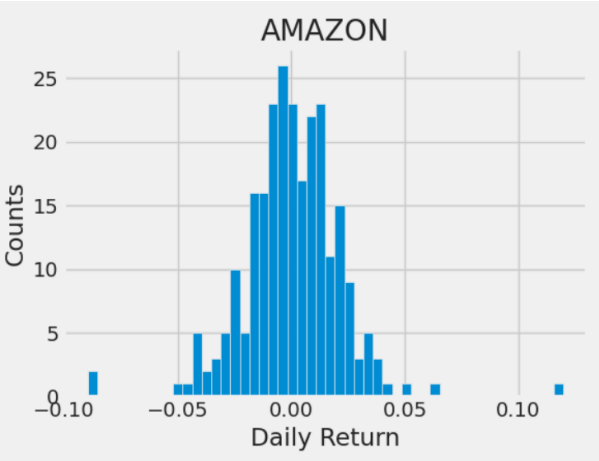
On the left, is a recreation of the paper's AMNZ average moving day closing cost. On the right, is the original. Again, everything matches besides extra data at the start of the x-axis.





On the left, is a recreation of the paper's AMNZ daily return plot. On the right, is the original. Everything seems to match, besides extra data at the start of the x-axis, and slight line thickness differences.





Last, on the left, is a recreation of the paper's AMNZ histogram plot of daily returns. On the right, is the original. This one was another shot in the dark trying to guess the exact bin count used for the original plot. The closest bin count that I found was 59. The extra data could affect the bin ratios. Overall, the AMZN model's evaluation didn't compare to the original findings, but the graphs were near perfect to those in the paper.

After finding our results, we compiled all of the reproducibility challenges faced for each ticker. The biggest issue we faced as a team was a resource issue. The code appendix supplied us with where to easily get ticker data as well as the model pipeline. However, the original paper did not provide us with the resources to easily collect or download the news data. In order for growing research in stock market analysis, researchers should provide an entire snapshot of their data, model, environment, and overall methodology. Even when we faced challenges in version requirements, randomness, and lack of key data, we still attempted to reproduce the original findings. Each ticker's reproducibility encountered its own issues.

<u>Apple's Ticker Challenges & Considerations:</u>

For Apple (AAPL) stock, several challenges were encountered during the reproduction process that impacted the ability to fully replicate the original results. One of the primary challenges was the lack of access to the original sentiment data used in the study. The paper states that sentiment scores were derived from financial news and social media sources, but it does not provide the actual dataset, APIs used, or detailed instructions for collecting this data. As a result, the reproduction relied on neutral sentiment values, which significantly reduced the model's ability to capture market sentiment and likely contributed to the higher error metrics observed.

Another major challenge was the lack of detailed information regarding the computational environment and implementation specifics. While the code appendix provided a general pipeline, it did not include exact package versions, dependency configurations, or complete reproducibility instructions. During implementation, issues such as TensorFlow installation errors, import conflicts, and version compatibility problems were encountered and had to be resolved manually. These environment-related issues demonstrate how sensitive deep learning models are to software configurations and highlight the importance of fully documenting experimental setups.

Additionally, there were ambiguities in the data preprocessing steps. Although the paper describes normalization and sequence generation, it does not fully specify how certain aspects were handled, such as exact date alignment, potential differences in data ranges, and integration of sentiment with price data. These missing details make it difficult to ensure that the reproduced dataset matches the original, which can significantly impact model performance.

<u>Microsoft's Ticker Challenges & Considerations:</u>

For Microsoft (MSFT) stock, a key limitation encountered during this process was the news API's limited historical coverage. Despite requesting sentiment data for the entire timeframe (April 2024 to April 2025), the API only returned articles for a small subset of recent dates: March 25, 2025; March 26, 2025; March 27, 2025; March 28, 2025; and March 31, 2025. No sentiment data was available for most of the earlier period. Consequently, missing sentiment values were set to a neutral value of 0, resulting in a dataset where most days lacked any sentiment signal. This significantly reduced the model's reliance on sentiment, causing it to behave more like a price-only LSTM.

The reproduction attempt faced several technical and methodological challenges. One major issue was environmental compatibility; the original paper did not specify exact software versions, leading to inconsistencies during model execution. To address this, the implementation was completed using Python version 3.12.9 (February 2025), ensuring compatibility with TensorFlow, NumPy, and other dependencies. This highlights a common reproducibility issue: missing environmental details can significantly impact results. Additionally, library version mismatches initially caused errors during model training and data preprocessing, necessitating adjustments to import paths and package installations.

Another significant challenge was the limitation of the sentiment API. While the paper described integrating sentiment from real-time news and social media streams, the free API key used for this ticker returned only a small number of articles and did not provide comprehensive historical coverage across the entire dataset period. This resulted in a sparse sentiment feature, where

most dates contained neutral values. As a consequence, the model relied more on historical price data than on sentiment, thereby diminishing the sentiment component's overall impact compared to the original study.
In summary, reproducing the Microsoft stock prediction demonstrated that while the LSTM-based framework can be effectively implemented, achieving identical results is challenging due to missing details, API limitations, and environmental differences. This highlights broader reproducibility challenges in data science, where incomplete documentation and resource constraints can lead to variations in outcomes.

Amazon's Ticker Reproducibility:

For Amazon (AMZN) stock, the first major hurdle was simply the location on my device where I was building my environment. All my schoolwork is saved under a specific folder on my ProtonDrive (kind of like OneDrive but doesn't sell your data). When building the venv for AMZN's reproduction, pip kept corrupting and disappearing. Due to TensorFlow's big pip wheel, ProtonDrive did not seem to enjoy it and would take too long to sync the files. The solution to this was moving it off the cloud sync and onto a regular mounted drive.

The second challenge faced was the Python library versions. For the main packages, I based my version on existing requirements.txt files I found on random GitHub repositories that utilized Python v3.12.9 which helped a lot. But the library yfinance caused me a lot of issues when trying to get AMZN stock ticker data. Originally, I encountered an error that I couldn't find on Google for the package version I had. Ultimately, it was my VPN. The package uses Yahoo Finance's API. Their API did not like users who use a VPN so all my requests were rejected. Then I faced a new issue, the library would return no data. The version I was using was for February 2025 but Yahoo Finance had a major update to their service a month or two after. After switching to the latest yfinance version, I was able to successfully get the ticker data needed.

Last, the third issue faced was a lack of news data resources. I first tried a Python library newsapi-python, which has a free API key tier. This library lets you search for top news headers with specific key words ("AMZN" and "Amazon") from different sources and dates. Because it's a free tier, they only let you query articles that are a month old. In our case we need articles from over a year ago. They also limit to 100 requests a day, but we need data for 267 total days. The next tier which lets you query articles up to 5 years and 250,000 requests a month costs $449… nope. The next solution I found was an API called Alpha Vantage. This one lets you query articles over a year old but comes with limitations as well. The free tier only lets you send 10 requests a day. In order to query 267 requests, it would cost $149.00… nope. With no other options besides collecting the data myself, I decided to only use their example header that was provided in the code appendix. This of course gave a zero sentiment score, leading to no reproducibility with the AMZN model evaluations.

In conclusion, each ticker had its own challenges, resource considerations, and integrity implications. The original authors should have shared the full code, data, and computing environment that they used during their research. With no source code, like GitHub, we as reproducers weren't able to see the different experiments that the original authors used. They provided a code appendix, but this did not document the full picture as the date range provided

was incorrect. In order to improve this process in the future, all news headers, randomness seeds, environment versions, and more code documentation is needed to be referenced. Without it, future researchers will struggle to fill gaps when it comes to enhancing market predictions with deep learning frameworks.

Citations

- Advanced Stock Market Prediction Using Long Short-Term Memory Networks: A Comprehensive Deep Learning Framework: arXiv:2505.05325 [cs.CE].